

บทที่ 3

Enhancing Rich Client Communications with the Microsoft* Real-Time Communications API

3.1 Introduction

ในส่วนแรก “การรวมการสื่อสารของ client หลายๆตัวด้วย [Microsoft Real-Time Communications API](#)” และอธิบายถึงตัวอย่างว่าทำอะไรและ ความสามารถโดยรวมของมัน เหมือน audio/video conferencing, instant messaging และ application sharing ใน application ที่คุณใช้ Real-Time Communications (RTC) Client API ในส่วนนี้อธิบายเพิ่มเติมถึง เหตุผลต่างๆถึง การสร้างการสื่อสารว่าทำอะไร โดยใช้ RTC Client API ในการเพิ่มแต่ละความสามารถที่แสดง ข้อมูล profile และ buddy list คุณจะได้เรียนรู้ขั้นตอนการใช้งานนี้ โดยใช้ Microsoft* Windows* XP application created ในส่วนแรก

Microsoft's real-time communication (RTC) API ได้ทำการ encapsulates การสื่อสารแบบใหม่ของ PC based ที่น่าสนใจเอาไว้ RTC API enables พัฒนาให้ ความสามารถในการสื่อสาร ของ Microsoft Windows XP* based application ใดก็ตามดีขึ้น มันทำให้การพัฒนาด้วยการสร้างบล็อก ในการ add instant messaging, voice และ video-conferencing, และ application sharing/collaboration capabilities to their application สำหรับโปรเซส ที่เพิ่มเข้าไปจำนวนมากของ ความสามารถที่ใช้ในการสื่อสารในการที่ แอปพลิเคชัน มิให้ใช้ของ RTC API โดยตรง อันดับแรก แอปพลิเคชันต้องกำหนดความสามารถของ platform การสื่อสารของไคลเอนท์อีกความสารถหนึ่ง ของ platform ที่รู้คือ แอปพลิเคชันสามารถเลือกว่า ชอบ audio และ video devices ที่ใช้ระหว่างส่วน ของการสื่อสาร แอปพลิเคชันที่พร้อมจะเริ่มส่วนของสื่อสารด้วยตัวอื่นหลังจากนั้น ส่วนที่สร้าง และเพิ่มเข้าไปในส่วนของการสื่อสารและ client application coordinates จะทำการ capture, compression, และ transmission ข้อมูลจนกระทั่งฝั่งรับและผู้รับ decompresses, และ replays the transmitted data

3.1.1 RTC Client Initialization & Platform Capabilities

ขั้นตอนแรก RTC client interface ต้องเพิ่มไปยังแอปพลิเคชันซึ่งทำการอินเทอร์เน็ตเฟสเข้ามาได้โดย CoCreateInstance() using the CLSID_RTCCClient (GUID = {7a42ea29-a2b7-40c4-b091-f6f024aa89be}) ซึ่ง อินเทอร์เน็ตเฟสหนึ่งที่ยอมให้ใช้และเป็นไปได้ ที่ใช้ในการกำหนด communication session capabilities ของแพลตฟอร์ม

```
// Initialize the RTC COM object
hr = CoCreateInstance (CLSID_RTCCClient, NULL,
    CLSCTX_INPROC_SERVER, IID_IRTCCClient,
    (LPVOID *)&m_pClient);

// Initialize the client interface
hr = m_pClient->Initialize();
```

เมื่อความสามารถของแพลตฟอร์มที่รู้จักคือ ชนิดที่อ้างถึงของการสื่อสาร และ อุปกรณ์เสริม(camera, microphone) ที่ต้องการเลือก มันเป็นสิ่งสำคัญที่ต้องจำถ้าต้องการ communication modes ที่มันไม่ยอมให้ใช้บน participant's platform, และ RTC client dynamically กำหนดโดย ชนิดของการสื่อสารที่ใช้สำหรับตัวอย่างทุกชนิดของการสื่อสารคือทางเลือกที่ดีที่สุด นี่คือตัวที่บอกว่าผู้ที่อยู่ใน communication session สามารถ share applications, transmit instant messages, audio, or video เมื่อยอมให้ใช้ฟังก์ชันนั้น

```
m_pClient->SetPreferredMediaTypes ( RTCMT_ALL, VARIANT_TRUE );
```

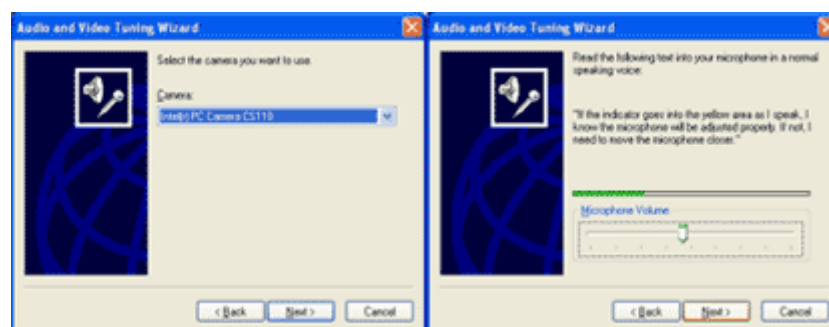
คงยังอีกนานที่ codec จะใช้ได้โดย window RTC client ในการส่ง audio และ video data ระหว่าง session participants ที่ขึ้นอยู่กับความสามารถของแพลตฟอร์ม ของคู่ที่เราทำการติดต่อใน session และ bandwidth ที่ใช้ระหว่างการสื่อสาร สำหรับ video ซึ่ง Windows RTC client รองรับ H.261 และ H.263 codecs ที่ QCIF (176x144) resolution ซึ่ง bit rate ที่ใช้ในการส่ง video data อยู่ในช่วงระหว่าง 6 และ 125-KBps มันเป็นไปได้ที่จะเกิดจากกระยะทางและ วิธีการในทางปฏิบัติของ การส่ง video ของ IRTCCClient interface methods MaxBitRate และ put_TemporalSpatialTradeOff.

Windows RTC client จะรองรับ audio codec หลายๆตัว,การส่งใหม่ และการเลือก audio portion สำหรับ communication session ก็คือพื้นฐานบน capabilities ของส่วนผู้ใช้ใน session ซึ่งค่า default codec ที่ใช้ใน voice communications คือ SIREN codec ซึ่งตารางของ audio codec ที่รองรับโดย Windows RTC client ได้แสดงดังตารางข้างล่าง

CODEC	Sampling Rate (Khz)	Bit Rate (Kbps)	Frame Size (msec)
G.711	8	64	20
G.722.1	16	24	20
G.723	8	6.4	30, 60, or 90
GSM	8	13	20
DVI4	8	32	20
SIREN	16	16	20, or 40

ตารางที่ 2 ตารางของ audio codec ที่รองรับโดย Windows RTC client

การที่จะทำการปรับค่าอุปกรณ์การสื่อสารใดๆ หลังจากทำการ preferred communication types และได้เลือก associated devices ซึ่ง RTC API จะเตรียม communication device tuning wizard ที่จะให้ผู้ใช้งานทำการส่งรุ่นที่เลือกให้กับ dialogs เพื่อให้ช่วยหาและปรับกล้องหรือไมโครโฟนในการร้องขอ tuning wizard ใช้ method InvokeTuningWizard()ของ RTCCClient interface.



รูปที่ 3.1.1 แสดง wizard เมื่อมีการเรียกใช้ method InvokeTuningWizard()

3.2 Get Ready

อธิบายถึงโครงสร้างของตระกูล XML Schema ที่อ้างถึงในบทแรก สำหรับ information เริ่มต้นของ RTC object สำหรับ source code ที่มากับบัพนี้มีสองเวอร์ชันคือ: [RTCSample.exe](#)* เป็นตัวอย่าง code ที่แสดงทุกการทำงานของ ฟังก์ชัน ที่มีให้ของ enabling audio/video conferencing, instant messaging, presence และ buddy list [AVDConf2.exe](#) ตัวอย่าง code ที่มีอยู่ใน capabilities ที่เป็น RTCSample.exe ตัวอย่าง code ด้วยการรวม features แต่ละตัวเช่น whiteboard sharing, application sharing, Intel® Pentium® 4 processor ด้วย Hyper-Threading Technology¹ detection, และ CPU utilization meter

3.2.1 สิ่งที่ต้องการ:

Microsoft* Visual Studio* 6.0 with SP5

Microsoft Platform SDK

SIP Server or Microsoft Windows* .NET Server Beta 3

3.3 Session Initiation Protocol Server

เป็นการรวมเอา presence , profile และ buddy list เข้าไปในแอปพลิเคชันของคุณว่าคุณต้องการ SIP server หรือ Microsoft Windows .NET Server Beta 3 สำหรับ SIP registrar server จะต้องการเมื่อประสงค์จะแสดงข้อมูลและต้องการใช้งานส่วนของแต่ละ Buddy List ผู้ใช้สามารถจิสแต่ละ presence information ด้วยเซิร์ฟเวอร์ และ ค้นหา presence information ด้วยบริการนี้ เซิร์ฟเวอร์จะใช้ Session Initiation Protocol (SIP) และอ้างถึง protocol SIMPLE ที่อยู่ภายใต้ โพรโตคอลการสื่อสาร ซึ่ง SIP จะเตรียมการรองรับได้ดีมากสำหรับการสื่อสารแบบ multi-modal ซึ่ง SIP และ SIMPLE คือโพรโตคอลที่ทำได้มากกว่า text messaging sharing มันสามารถจัดการ manage voice, video, application sharing และอีกหลายอย่าง SIP protocol initiates sessions ในไอพีเน็ตเวิร์คและ registers presence information สามารถทำได้สองทางคือ PC-to-PC communication หรือ collaborative multi-media conference session ซึ่ง SIP คือ Internet Engineering Task Force (IETF) signaling protocol สำหรับ establishing, manipulating, และ tearing down sessions ซึ่ง จุดประสงค์หลักของ SIP คือช่วยรับการเริ่มต้นของ potential session participants เมื่อมันทำการ SIP มีรายละเอียดที่ "simple, extensible" IP telephony signaling protocol

3.3.1 การเริ่มต้นสำหรับ Presence และ Buddy List Notifications

การรับ event notifications ใหม่สำหรับ presence, buddy list, และ profiling จะกำหนดตาม event ตาม event filter masks เพื่อให้แอปพลิเคชันของคุณสามารถรับ event notification จาก RTC เลเยอร์ได้

```
RTCEF_REGISTRATION_STATE_CHANGE 0x00000002
#define RTCEF_BUDDY 0x00000100
#define RTCEF_WATCHER 0x00000200
#define RTCEF_PROFILE 0x00000400
```

ตัวอย่างอีกอันหนึ่งในการ register ทุก event คือการใช้ RTCEF_ALL macro notify ซึ่ง RTC เลเยอร์จะส่งทุก event ไปยังแอปพลิเคชัน

```
long lEventMask = RTCEF_ALL;
```

3.3.2 Handling RTC Events

ตาม code ด้านแสดงให้เห็น additional events ในการโปรเซสเมื่อ registering ไปยัง SIP server ด้วย user profile ,ส่งและรับ presence information และ buddy list events ที่แต่ละ event คือ received , event filter method จะใช้การแยก RTC interface ไปยังโปรเซสในการทำ received event

```
HRESULT CAVDConfDlg::OnRTCEvent(UINT message, WPARAM wParam,
LPARAM lParam)
{
    IDispatch * pDisp = (IDispatch *)lParam;
    RTC_EVENT enEvent = (RTC_EVENT)wParam;
    HRESULT hr;
    // Based on the RTC_EVENT type, query for the
    // appropriate event interface and call a helper
    // method to handle the event
    switch ( wParam )
    {
        ...
        case RTCE_REGISTRATION_STATE_CHANGE:
```

```

{
    IRTCRegistrationStateChangeEvent * pEvent = NULL;
    // Get the event handle associated with the current session.
    hr = pDisp->QueryInterface(
IID_IRTCSessionStateChangeEvent,
(void *)&pEvent );
    if (SUCCEEDED(hr))
    {
        OnRTCSessionStateChangeEvent(pEvent);
        SAFE_RELEASE(pEvent);
    }
}

break;

case RTCE_BUDDY:
{
    IRTCBuddyEvent * pEvent = NULL;
    hr = pDisp->QueryInterface(
IID_IRTCBuddyEvent,
(void *)&pEvent );
    if (SUCCEEDED(hr))
    {
        OnRTCBuddyEvent(pEvent);
        SAFE_RELEASE(pEvent);
    }
}

break;

case RTCE_WATCHER:
{
    IRTCWatcherEvent * pEvent = NULL;
    hr = pDisp->QueryInterface( IID_IRTCWatcherEvent,
(void *)&pEvent );
    if (SUCCEEDED(hr))

```

```

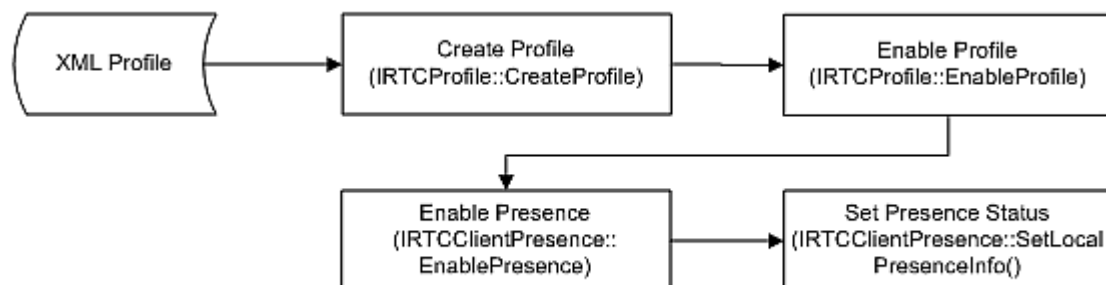
{
    OnRTCWatcherEvent(pEvent);
    SAFE_RELEASE(pEvent);
} }
break;
}

...
}

```

3.3.3 Enabling and Detecting Presence

Presence information service ที่มีให้ผู้ใช้แทรกไปยัง contact's presence status , บอก contacts เหล่านั้นหรือ สถานะ, และ เรียกกลุ่มสนทนาเหล่านั้น(or contacts) เพื่อส่งไปยัง registrar server ที่รองรับ location information ปัจจุบันของcontacts นั้นซึ่ง location สามารถเป็นได้ทั้ง PC หรือ โทรศัพท์และในอนาคตจะเป็น เซลล์โฟน , เพจเจอร์ หรือ handheld device ตามไดอะแกรมแสดง high-level steps ในการ register ของไคลเอนท์บนซีพเซิร์ฟเวอร์และ enable Presence



รูปที่ 3.3.1 แสดงขั้นตอนการทำ Enable Profile และ Presence service

3.3.4 SIP Server Registration

ในการทำให้ Presence service ใช้งานได้คุณต้องสร้าง Profile object สร้างโดยใช้

[IRTCClientProvisioning::CreateProfile](#) *method ในการสร้าง Profile object, client application

จะต้องสร้าง XML string ที่ตรงกับข้อกำหนดของ schema สำหรับ Attributes ของ XML schema ประกอบด้วย:

Provision settings – unique identifier สำหรับ Profile

User settings – User's URI, realm, และ login account information

Client settings – Information เกี่ยวกับ client application ที่ไม่ได้อ้างอิงถึง communications link คือ ส่วนของ information ที่เลือกได้

Provider settings – Information เกี่ยวกับ Internet telephony service provider (ITSP).

SIP server settings – ขั้ระบุถึง SIP servers available ที่มีให้ ,หน้าที่ของ SIP server และชนิดของ session server ที่รองรับ

Register ที่ใช้บน SIP server และ client ต้องสร้าง XML profile string เริ่มต้นให้กับ instruction ของ RTC client API ในการสื่อสารไปยัง SIP server สำหรับตัว XML string ที่สร้าง เรียก IRTCClientProvisioning::CreateProfile() method ไปยังการสร้าง profile object ขั้นตอนต่อมาเรียก IRTCClientProvisioning::EnableProfile() method เป็นการ register ให้ผู้ใช้บน RTC server และกำหนดชนิดของ session ที่ profile ทำการ register บน server

ชนิด registration คือ: การยอมให้เข้ามาของ PC-to-PC sessions

(RTCRF_REGISTER_INVITE_SESSIONS), ยอมให้ instant messaging session เข้ามา

(RTCRF_REGISTER_MESSAGE_SESSIONS),ยอมให้ watchers เข้ามา

(RTCRF_REGISTER_PRESENCE),หรือยอมให้ทุก registration types

(RTCRF_REGISTER_ALL). Setting RTCRF_REGISTER_PRESENCE หรือ

RTCRF_REGISTER_ALL จะแจ้ง registrar server ให้ client accepts SIP SUBSCRIBE method.

นี่คือสิ่งที่ทำให้ผู้ใช้สำหรับเปลี่ยน presence status , get notifications ของการเปลี่ยน contact's presence status และคนอื่นที่เพิ่มเข้ามาใน Watcher list

watcher objects จะให้ status ของ user ใน buddy list ซึ่ง watcher สามารถเรียงลำดับ สำหรับ status ของ Presence entity และ notified of changes ใน status ของ user ใน buddy list ซึ่ง watcher ทำได้ทั้ง block หรือ allow จาก status ของ Presence entity เหล่านั้น

ตัวอย่าง code ที่แสดงให้เห็นขั้นตอน register user และ enable presence และ buddy list.

```
HRESULT CAVDConfDlg::DoSIPLogin(BSTR bXMLObj)
{
    HRESULT hr;
    ...
    // Get the RTC client provisioning interface
    IRTCCClientProvisioning * pProv = NULL;
    hr = m_pClient->QueryInterface(
        IID_IRTCCClientProvisioning,
        (void *)&pProv);
    if (FAILED(hr))
    {
        // Query Interface
        failed
        return hr;
    }

    // Create a RTC profile object from the XML
    // provisioning document
    hr = pProv->CreateProfile(bXMLObj,
        &m_pProfile);
    SAFE_FREE_STRING(bXMLObj);
    if (FAILED(hr))
    {
        // CreateProfile failed
        SAFE_RELEASE(pProv);
        return hr;
    }

    // Enable the RTC profile object
    hr = pProv->EnableProfile(m_pProfile,
        RTCRF_REGISTER_ALL);
    SAFE_RELEASE(pProv);
}
```

```

if (FAILED(hr))
{
    // EnableProfile failed

    return hr;
}

// Enable presence

// It is best to enable presence immediately after enabling
the
// profile so that incoming watchers will not be lost.

    // See below for EnablePresence() method
below for function definition
hr = EnablePresence(TRUE);
if (FAILED(hr))
{
    // DoEnablePresence failed

    DoSIPLogoff(); // See code sample
AVDConf2.exe for function sample

    return hr;
}

    return S_OK;
}

```

นี่คือ profile ที่ enable และคุณต้อง enable presence ป้องกันการสูญเสียการเข้ามาของ watcher event ซึ่ง EnablePresence() method แสดงว่าสร้าง buddy list อย่างไร

```

HRESULT CAVDConfDlg::EnablePresence(BOOL bEnable)
{
    IRTCCClientPresence * pPresence = NULL;

    HRESULT hr;

    // Cleanup the buddy list

    ClearBuddyList();

    // Get the RTC client presence interface

```

```

hr = m_pClient->QueryInterface(
IID_IRTCCClientPresence,
(void *)&pPresence);
if (FAILED(hr))
{
// QueryInterface failed
return hr;
}

// Get the location of the presence storage
VARIANT varStorage;
VariantInit(&varStorage);
varStorage.vt = VT_BSTR;
varStorage.bstrVal = SysAllocString(L"presence.xml");

// If we are disabling presence, save the latest
// copy of the presence data to presence.xml file.
if (!bEnable &&
m_bPresenceEnabled)
{
hr = pPresence->Export(varStorage);
if (FAILED(hr))
{
// Export failed
SAFE_RELEASE(pPresence);
VariantClear(&varStorage);
return hr;
}
}

// Enable presence
hr = pPresence->EnablePresence(

bEnable ? VARIANT_TRUE : VARIANT_FALSE, varStorage);
VariantClear(&varStorage);

```

```

if (FAILED(hr))
{
    // EnablePresence failed
    SAFE_RELEASE(pPresence);
    return hr;
}

// Set the enabled flag
m_bPresenceEnabled = bEnable;

// If we are disabling presence, cleanup the
// presence data
if (!bEnable)
{
    // Cleanup buddies
    IRTCEnumBuddies * pEnumBuddy = NULL;
    IRTCBuddy * pBuddy = NULL;

    if (FAILED(hr))
    {
        // Enumerate buddies failed
        SAFE_RELEASE(pPresence);
        return hr;
    }

    // Enumerate user's buddy list.
    hr =
pPresence->EnumerateBuddies(&pEnumBuddy);
    if (FAILED(hr))
    {
        // Enumerate buddies failed
        SAFE_RELEASE(pPresence);
        return hr;
    }

    while (pEnumBuddy->Next(1, &pBuddy, NULL)
== S_OK)

```

```

{
    pPresence->RemoveBuddy(pBuddy);
    SAFE_RELEASE(pBuddy);
}

SAFE_RELEASE(pEnumBuddy);

// Cleanup watchers
IRTCEnumWatchers * pEnumWatcher = NULL;
IRTCWatcher * pWatcher = NULL;
hr = pPresence->EnumerateWatchers(&pEnumWatcher);
if (FAILED(hr))
{
    // Enumerate watchers failed
    SAFE_RELEASE(pPresence);
    return hr;
}

while (pEnumWatcher->Next(1, &pWatcher,
NULL) == S_OK)
{
    pPresence->RemoveWatcher(pWatcher);
    SAFE_RELEASE(pWatcher);
}

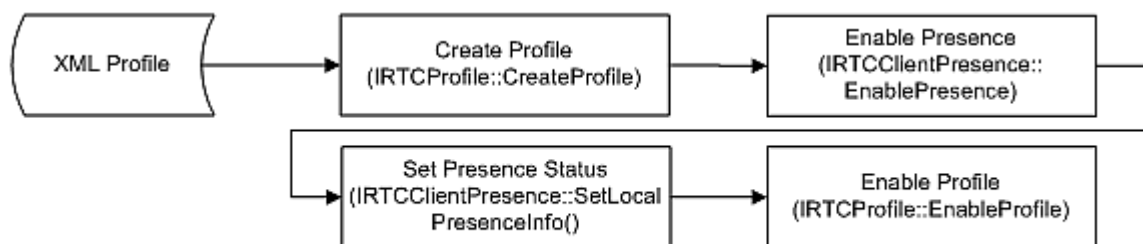
SAFE_RELEASE(pEnumWatcher);
}

SAFE_RELEASE(pPresence);
return S_OK;
}

```

ประกาศใน parameter ที่สองของ pPresence->EnablePresence() function และร้องขอ variant storage ไปเก็บใน buddy list และ allowed and blocked watcher lists ซึ่ง variant storage สามารถอยู่ในรูปของ file name (BSTR), [DOMDocument*](#) object, หรือ object ใดๆ ที่ supports [IStream*](#), [ISequentialStream*](#), หรือ [IPersistStream*](#) หลังจาก register ด้วย RTC server และแอปพลิเคชันรับ RTCE_REGISTRATION_STATE_CHANGE event, ด้วยการแสดงว่าการ ลงทะเบียนสำเร็จ

หรือ ล้มเหลว Presence service สามารถ enabled ก่อน registering Profile ของคุณบนเซิร์ฟเวอร์ซึ่งไดอะแกรมข้างล่างแสดงว่าทำอย่างไร Presence service จึงสามารถ enabled ก่อน register ได้ นี่คือการ enable ที่ผู้ set status ก่อน register บน server ซึ่ง user สามารถ register ใหม่บน SIP server แต่สามารถแสดง "Off-line" ที่ Watchers และเห็น contacts' status ของเขาได้ซึ่ง feature นี้เป็นประโยชน์มากเมื่อ user ต้องการ monitor group's presence status โดยไม่มี intrusive



รูปที่ 3.3.2 แสดงขั้นตอนการทำ Enable Presence หลังจาก registration

Note the CreateProfile() method สามารถเรียกหลังจาก setting the Presence status.

ตาม XML Schema

```

<?xml version="1.0" ?>
<Schema name="ProvisioningSchema.xml"
xmlns="urn:schemas-microsoft-com:xml-data"
xmlns:dt="urn:schemas-microsoft-com:datatypes">
<comments>Schema Version MajorRevisionNumber = 1
MinorRevisionNumber = 0
</comments>
<ElementType name="provision" model="closed">
<AttributeType name="key" required="yes" />
<attribute type="key" />
<AttributeType name="name" required="yes" />
<attribute type="name" />
<AttributeType name="expires" dt:type="dateTime.tz"
/>
<attribute type="expires" />
<element type="user" minOccurs="1" maxOccurs="1"
  
```

```

/>
<element type="sipsrv" minOccurs="1" maxOccurs="*"
/>
<element type="client" minOccurs="0" maxOccurs="1"
/>
<element type="provider" minOccurs="0" maxOccurs="1"
/>
</ElementType>
<ElementType name="user" model="closed">
<AttributeType name="uri" required="yes" />
<attribute type="uri" />
<AttributeType name="account" required="no" />
<attribute type="account" />
<AttributeType name="name" required="no" />
<attribute type="name" />
<AttributeType name="password" required="no" />
<attribute type="password" />
<AttributeType name="realm" required="no" />
<attribute type="realm" />
</ElementType>
<ElementType name="client" model="closed">
<AttributeType name="name" required="yes" />
<attribute type="name" />
<AttributeType name="banner"
dt:type="bool"
required="no" />
<attribute type="banner" />
<AttributeType name="updates"
dt:type="bool"
required="no" />
<attribute type="updates" />
<AttributeType name="minver"

```

```

dt:type="fixed.14.4"
required="no"/>
<attribute type="minver" />
<AttributeType name="curver"

dt:type="fixed.14.4"
required="no"/>
<attribute type="curver" />
<AttributeType name="updateuri"
dt:type="uri"
required="no" />
<attribute type="updateuri" />
<element type="data" minOccurs="0" maxOccurs="1"
/>
</ElementType>
<ElementType name="data" model="open" />
<ElementType name="provider" model="closed"
content="mixed">
<AttributeType name="name" />
<attribute type="name" />
<AttributeType name="homepage" dt:type="uri" required="no"
/>
<attribute type="homepage" />
<AttributeType name="helpdesk" dt:type="uri" required="no"
/>
<attribute type="helpdesk" />
<AttributeType name="personal" dt:type="uri" required="no"
/>
<attribute type="personal" />
<AttributeType name="calldisplay" dt:type="uri"
required="no" />
<attribute type="calldisplay" />

```



```

    <AttributeType name="idledisplay" dt:type="uri"
required="no" />
    <attribute type="idledisplay" />
    <element type="data" />

</ElementType>
<ElementType name="sipsrv" model="closed"
content="mixed">
    <AttributeType name="addr" required="yes" />
    <attribute type="addr" />
    <AttributeType name="protocol"
dt:type="enumeration"
dt:values="TCP UDP TLS"
required="yes" />
    <attribute type="protocol" />
    <AttributeType name="auth"
dt:type="enumeration"
dt:values="basic digest"
required="no" />
    <attribute type="auth" />
    <AttributeType name="role"
dt:type="enumeration"
dt:values="proxy registrar"
required="yes" />
    <attribute type="role" />
    <element type="session" minOccurs="0" maxOccurs="*"
/>
</ElementType>
<ElementType name="session" model="closed">
    <AttributeType name="party"
dt:type="enumeration"
dt:values="first third" />

```

```

<attribute type="party" />
<AttributeType name="type"
dt:type="enumeration"
dt:values="pc2pc pc2ph ph2ph im" />

<attribute type="type" />
</ElementType>
</Schema>

```

3.3.5 การสร้าง XML Profile

นี่คือ code ที่แสดงการ create XML profile string.

```

HRESULT CSIPLogin::CreateXMLProvision(LPSTR
szURI, LPSTR szSIPIP,
    LPSTR
szTransport, BSTR
*bstrBuf)
{
    ...
    // Generate the XML provisioning document
    wsprintf(szBuf, "<provision key=\"AVDConf_2\"
name=\"AVDConf_2\">"
        "<user uri=\"%s\" account=\"\" password=\"\" realm=\"%s\" />"
        "<sipsrv addr=\"%s\" protocol=\"%s\" %s role=\"proxy\">"
        "<session party=\"first\" type=\"pc2pc\" />"
        "<session party=\"first\" type=\"pc2ph\" />"
        "<session party=\"first\" type=\"im\" />"
        "</sipsrv>"
        "<sipsrv addr=\"%s\" protocol=\"%s\" %s role=\"registrar\"/>"
        "</provision>",
        szURIBuf, szRealm,
        szSIPIP, szTransport, bBasicAuth ?

```

```

"auth=\"
basic\\\" : \"\",
    szSIPIP, szTransport, bBasicAuth ?
"auth=\"
basic\\\" : \"\"
);

... .
return S_OK;
}

```

นี่คือตัวอย่างแอปพลิเคชันและ code สั้นๆ ในส่วนที่แสดงข้างบน มันไม่จำเป็นต้องรวม account userid และ password ที่ไม่ต้องการที่ SIP server อย่างไรก็ตามถ้า SIP server ร้องขอ login account , userid และ password ที่สามารถรวมอยู่ใน Profile string ในการ login ไปยัง SIP server มีเพียงข้อมูลข่าวสารเท่านั้นที่ต้องการที่ user Uniform Resource Identifier (URI), realm หรือ domain, RTC server IP, authentication method, และ transport ที่ใช้ในการสื่อสารไปยัง เซิร์ฟเวอร์ที่รองรับการส่งข้อมูล คือ TCP, UDP, และ TLS ซึ่ง SIP server ต้องใช้รองรับทั้ง Basic และ Digest authentication ถ้า authentication คือ Basic และ transport ต้องมี TLS สำหรับความปลอดภัย

3.3.10 Creating the Buddy List และ Watcher Object

Profile ตัวหนึ่งคือ registered และ presence ที่ enabled, adding ให้กับผู้ใช้ใหม่ไปยัง buddy list คือ ตัวอย่าง ใช้ IRTCCClientPresence interface, ด้วยการ provides methods ให้กับ enable presence, add a buddy, remove a buddy, enumerate watchers, set local presence status, กำหนดอย่างไรใน application handles subscriptions จาก new Watchers, และ set the privacy mode นี่คือนี่คือ code ที่ไม่ใช่ demonstrate ,ว่ามีเครื่องมืออย่างไรบ้างใน privacy mode, แต่อย่างไรก็ตามมันคือนี่คือ feature ถ้า user create a discrete list ของสำหรับเรียก people permitted

```

// Get the RTC client presence interface
IRTCClientPresence * pPresence = NULL;
hr = m_pClient->QueryInterface(
IID_IRTCClientPresence,

```

```

(void *)&pPresence);
if (FAILED(hr))
{
    // QueryInterface failed

    char szBuf[256];

    wsprintf (szBuf, "Failed to
Query
Presence Interface\nErr = 0x%x", hr );

    MessageBox ( szBuf );

    return hr;
}

// Add the buddy
IRTCBuddy * pBuddy = NULL;
hr = pPresence->AddBuddy(
    bstrURI,
    bstrName,
    NULL,
    VARIANT_TRUE,
    NULL,
    0,&pBuddy);
SAFE_RELEASE(pPresence);
if (FAILED(hr))
{
    // Addbuddy failed

    SAFE_RELEASE(pBuddy);

    char szBuf[256];

    wsprintf (szBuf, "Failed to
Add
Buddy to List.\nErr = 0x%x", hr );

    MessageBox ( szBuf );

    return hr;
}

```

```
// Update the buddy list entry
UpdateBuddyList(pBuddy);
SAFE_RELEASE(pBuddy);
```

ถ้า new buddy สร้างสมบูรณ์ AddBuddy() method จะทำการรีเฟรชพอยน์เตอร์ไปยัง [IRTCBuddy*](#) interface บน created Buddy object ตัวใหม่ ใช้ [IRTCBuddy*](#) interface และ client application จะทำการ able ในการ get buddy's presentity URI, name ของ buddy, buddy's status, persistent type, และ private data รวมกับ buddy's presence.

3.3.11 Deregistering และ Disabling a Profile

การเรียก IRTCCClientProvisioning::DisableProfile() method ในการ deregister user จาก SIP server เพื่อให้แน่ใจว่า profile object มีการแก้ไขหลังจาก calling DisableProfile() method.

3.3.12 Optimizing for Performance

ใน article แรกเราแสดง CPU utilization เมื่อ running RTC application บน Pentium 4 processor-based system และยังคงจำนวนมากอยู่ของ headroom สำหรับ background tasks อย่างไรก็ตามการตอบรับของ application สามารถ hindered(ล่าช้า) โดยสถาปัตยกรรมของมัน สำหรับปัญหา address นี้, create threads ในการ run แบบ parallel และ service อื่นๆ ที่ต้องการกระทำที่เป็น immediate attention ซึ่ง Running multi-threaded application บน Pentium 4 processor ด้วย Hyper-Threading Technology1 และ Windows XP ด้วย Service Pack 1 (SP1) จะแสดง application's responsiveness ที่ดีขึ้นและ enable application multitask ที่เพิ่ม efficiently.

Windows XP รุ่น SP1 ตรวจสอบที่ Pentium 4 processor ด้วย Hyper-Threading Technology1 ที่ 2 logical processors, ซึ่ง Windows XP สามารถทำ schedule twice รวมของงานที่ทำที่ CPU ที่ใช้เปรียบเทียบกับ single logical CPU

3.3.8 Conclusion

Real-Time Communications (RTC) API จะทำให้คุณสร้าง functioning conferencing ได้เต็มที่และ collaboration tools, เมื่อเป็นการสื่อสารแบบ PC-to-PC, PC-to-Phone, หรือ Phone-to-Phone เราทดสอบให้เห็นใน article แรกว่ารวดเร็วอย่างไรคุณสามารถพัฒนา audio และ video conferencing application ด้วย instant messaging และ application sharing ใน article นี้เราเพิ่ม capabilities ในการรวม presence และ buddy list functions ในการสร้างการสื่อสาร และ track people's availability ทั้งคู่ต่อกันด้วย RTC API ด้วย Microsoft Real-Time Communications server, และคุณสามารถสร้าง sophisticated, effective corporate communication tools ที่จะเพิ่ม cross-site team productivity.

Communication applications ที่พัฒนาโดยใช้ RTC API และ run บน Pentium 4 Processor ด้วย Hyper-Threading Technology¹ และ Windows XP ที่เป็น SP1 จะได้ higher level ของ peer communications power ด้วยการตอบรับที่ดีขึ้นจนกระทั่ง performing multiple tasks concurrently